Machine Learning Algorithms

Every Data Scientist Must Know

Zamir Hussain
Department of Mathematics
University of Wah

February 9, 2026

Abstract

These notes bridge core linear algebra concepts with practical machine learning algorithms. Designed for mathematics students new to ML, each section provides intuitive explanations, concrete numerical examples, and clear connections to underlying mathematical principles. Emphasis is placed on "why this works" rather than formal proofs.

Contents

1 Introduction: The Linear Algebra Foundation

Intuition

Machine Learning = Mathematics + Data. At its core, every ML algorithm transforms data using matrices and vectors to find patterns. If you understand linear algebra, you already understand 80% of ML mathematics.

1.1 The Fundamental Equation

Most supervised learning problems can be framed as:

$$\mathbf{y} = f(\mathbf{X}\mathbf{w} + \mathbf{b})$$

where:

- $\mathbf{X} \in \mathbb{R}^{n \times d}$: Data matrix (n samples, d features)
- $\mathbf{w} \in \mathbb{R}^{d}$: Weight vector (what we learn)
- $\mathbf{b} \in \mathbb{R}$: Bias term
- $f(\cdot)$: Activation/transformation function
- $\mathbf{y} \in \mathbb{R}^{n}$: Predictions/targets

Linear Algebra Connection

Core Operations:

- **Dot Product:** $z = \mathbf{w}^\top \mathbf{x} = \sum_{i=1}^{d} w_i x_i$ (similarity measure)
- **Matrix Multiplication:** $\mathbf{Z} = \mathbf{X}\mathbf{W}$ (batch processing)
- **Norms:** $\|\mathbf{w}\|_p = (\sum |w_i|^p)^{1/p}$ (regularization)
- **Eigen Decomposition:** $\mathbf{A} = \mathbf{V}\mathbf{\Lambda}\mathbf{V}^{-1}$ (PCA, SVD)

2 Classification Algorithms

2.1 Logistic Regression: Probabilistic Classifier

Intuition

Think of logistic regression as "probability through a squashing function." It takes linear combinations of features and maps them to probabilities between 0 and 1 using the sigmoid function.

Linear Algebra Connection

Mathematical Foundation:

$$\text{Linear combination:} \quad z = \mathbf{w}^\top \mathbf{x} + b$$

$$\text{Sigmoid:} \quad \sigma(z) = \frac{1}{1 + e^{-z}}$$

$$\text{Loss (Cross-Entropy):} \quad J(\mathbf{w}) = -\frac{1}{n} \sum_{i=1}^{n} [y_i \log(\hat{y}_i) + (1 - y_i) \log(1 - \hat{y}_i)]$$

$$\text{Gradient:} \quad \nabla J(\mathbf{w}) = \frac{1}{n} \mathbf{X}^\top (\hat{\mathbf{y}} - \mathbf{y})$$

Step-by-Step Example

Spam Detection Example:

- Email features: [0,1,0,1,1] (1=word exists: "free", "money", "urgent", "click", "now")
- Trained weights: $\mathbf{w} = [0.5, -1.2, 0.3, 0.8, -0.5]$, $b = 0.1$
- Compute: $z = 0.5(0) + (-1.2)(1) + 0.3(0) + 0.8(1) + (-0.5)(1) + 0.1 = -0.8$
- Probability: $\sigma(-0.8) = \frac{1}{1+e^{0.8}} = 0.31$ (31% spam probability)

Why This Matters in ML

- **Interpretable:** Weights show feature importance
- **Probabilistic:** Outputs probabilities, not just classes
- **Foundation:** Building block for neural networks
- **Efficient:** Convex optimization guarantees global optimum

2.2 Naive Bayes: Conditional Independence Classifier

Intuition

"Naive" assumption: All features are independent given the class. Like calculating the probability of symptoms given a disease, assuming symptoms don't affect each other.

Linear Algebra Connection

Bayes Theorem with Independence:

$$P(y|\mathbf{x}) = \frac{P(y)P(\mathbf{x}|y)}{P(\mathbf{x})}$$

$$\text{Naive assumption:} \quad P(\mathbf{x}|y) = \prod_{i=1}^{d} P(x_i|y)$$

$$\text{Decision rule:} \quad \hat{y} = \arg\max_y P(y) \prod_{i=1}^{d} P(x_i|y)$$

Types:

- Gaussian NB: Continuous features $\sim \mathcal{N}(\mu, \sigma^2)$
- Multinomial NB: Count data (text)
- Bernoulli NB: Binary features

Step-by-Step Example

Weather Classification:

Outlook	Temp	Humidity	Windy	Play?
Sunny	Hot	High	False	No
Sunny	Hot	High	True	No
Overcast	Hot	High	False	Yes
Rain	Mild	High	False	Yes
Rain	Cool	Normal	False	Yes

Classify: [Rain, Hot, High, False]

$$\begin{aligned} P(\text{Yes}) &= 3/5 = 0.6 \\ P(\text{Rain}|\text{Yes}) &= 2/3 = 0.67 \\ P(\text{Hot}|\text{Yes}) &= 1/3 = 0.33 \\ P(\text{High}|\text{Yes}) &= 2/3 = 0.67 \\ P(\text{False}|\text{Yes}) &= 3/3 = 1.0 \\ P(\text{Yes}|\mathbf{x}) &\propto 0.6 \times 0.67 \times 0.33 \times 0.67 \times 1.0 = 0.089 \end{aligned}$$

2.3 K-Nearest Neighbors (KNN): Instance-Based Learning

Intuition

"Birds of a feather flock together." Classify new points based on majority vote of their k closest neighbors in feature space.

Linear Algebra Connection

Distance Metrics (all are norms):

$$\begin{aligned}\text{Euclidean:} \quad & d(\mathbf{x},\mathbf{y}) = \|\mathbf{x}-\mathbf{y}\|_2 = \sqrt{\sum (x_i - y_i)^2} \\ \text{Manhattan:} \quad & d(\mathbf{x},\mathbf{y}) = \|\mathbf{x}-\mathbf{y}\|_1 = \sum |x_i - y_i| \\ \text{Minkowski:} \quad & d(\mathbf{x},\mathbf{y}) = (\sum |x_i - y_i|^p)^{1/p}\end{aligned}$$

Algorithm:

1. Compute distance matrix between all points
2. For each test point, find k smallest distances
3. Majority vote among k neighbors

Step-by-Step Example

Classify point (6,5) with k=3:

Point	Features	Class
A	(1,2)	Red
B	(2,3)	Red
C	(5,6)	Blue
D	(6,7)	Blue

Distances from (6,5):

$$\begin{aligned}d(A) &= \sqrt{(6-1)^2 + (5-2)^2} = \sqrt{25+9} = \sqrt{34} \approx 5.83 \\ d(B) &= \sqrt{(6-2)^2 + (5-3)^2} = \sqrt{16+4} = \sqrt{20} \approx 4.47 \\ d(C) &= \sqrt{(6-5)^2 + (5-6)^2} = \sqrt{1+1} = \sqrt{2} \approx 1.41 \leftarrow \text{1st} \\ d(D) &= \sqrt{(6-6)^2 + (5-7)^2} = \sqrt{0+4} = 2.00 \leftarrow \text{2nd}\end{aligned}$$

Nearest: C (Blue), D (Blue), B (Red) $\rightarrow$ Majority: Blue

2.4 Support Vector Machine (SVM): Maximum Margin Classifier

Intuition

Find the widest possible "street" (margin) between classes. Like drawing the fattest possible line between two groups of points.

Linear Algebra Connection

Mathematical Formulation:

$$\text{Goal:} \quad \min_{\mathbf{w},b} \frac{1}{2}\|\mathbf{w}\|^2$$

$$\text{Subject to:} \quad y_i(\mathbf{w}^\top \mathbf{x}_i + b) \geq 1, \forall i$$

$$\text{Dual Form (Kernel Trick):} \quad \max_{\alpha} \sum \alpha_i - \frac{1}{2} \sum \sum \alpha_i \alpha_j y_i y_j K(\mathbf{x}_i, \mathbf{x}_j)$$

Kernels (implicit high-dimensional mapping):

- Linear: $K(\mathbf{x}, \mathbf{z}) = \mathbf{x}^\top \mathbf{z}$
- Polynomial: $K(\mathbf{x}, \mathbf{z}) = (\mathbf{x}^\top \mathbf{z} + c)^d$
- RBF: $K(\mathbf{x}, \mathbf{z}) = \exp(-\gamma \|\mathbf{x} - \mathbf{z}\|^2)$

Step-by-Step Example

Simple 2D Case:

Point	(x1,x2)	y
1	(1,1)	+1
2	(2,2)	+1
3	(3,1)	-1
4	(4,2)	-1

Optimal hyperplane: $x_1 - x_2 = 0$ (45° line)

- Weight vector: $\mathbf{w} = [1, -1]$, $b = 0$
- Margin width: $\frac{2}{\|\mathbf{w}\|} = \frac{2}{\sqrt{2}} = \sqrt{2}$
- Support vectors: Points 2 and 3 (closest to boundary)

2.5 Decision Tree: Hierarchical If-Else Rules

Intuition

Like playing 20 Questions: Ask a series of yes/no questions to classify data. Each question splits the data into purer subsets.

Linear Algebra Connection

Splitting Criteria:

$$\text{Gini Impurity:} \quad I_G(p) = 1 - \sum_{i=1}^{C} p_i^2$$

$$\text{Entropy:} \quad H(p) = -\sum_{i=1}^{C} p_i \log_2 p_i$$

$$\text{Information Gain:} \quad IG = H_{\text{parent}} - \sum \frac{|D_j|}{|D|} H(D_j)$$

One-Hot Encoding: Categorical features → Binary vectors
Example: Color ∈ Red, Green, Blue → [1,0,0], [0,1,0], [0,0,1]

Step-by-Step Example

Should We Play Tennis?

Outlook	Temp	Humidity	Windy	Play?
Sunny	Hot	High	Weak	No
Sunny	Hot	High	Strong	No
Overcast	Hot	High	Weak	Yes
Rain	Mild	High	Weak	Yes
Rain	Cool	Normal	Weak	Yes

Split on Outlook:

- Sunny: 2 No / 0 Yes → Gini = 0
- Overcast: 0 No / 1 Yes → Gini = 0
- Rain: 0 No / 2 Yes → Gini = 0

Perfect split! Information gain = max.

2.6 Random Forest: Wisdom of the Crowd

Intuition

Many decision trees voting together. Each tree sees slightly different data and features, reducing overfitting through collective wisdom.

Linear Algebra Connection

Bootstrap Aggregating (Bagging):

$$\text{Sample with replacement:} \quad D_i \sim \text{Bootstrap}(D)$$

$$\text{Train trees:} \quad T_i = \text{Train}(D_i, \text{random features})$$

$$\text{Aggregate:} \quad \hat{y} = \text{mode}\{T_i(\mathbf{x})\}$$

Out-of-Bag Error: Natural validation using unsampled data for each tree.

</> Step-by-Step Example

5-Tree Forest Prediction:

Tree1	Tree2	Tree3	Tree4	Tree5	Majority	Final
A	B	A	A	B	A(3), B(2)	**A**
B	B	A	B	B	A(1), B(4)	**B**
A	A	A	B	A	A(4), B(1)	**A**

Key advantages:

- Tree1 might overfit noise $\rightarrow$ Others compensate
- Different trees focus on different features
- Final prediction more robust

3 Regression Algorithms

3.1 Linear Regression: The Foundation

Intuition

Draw the "best fit" line through data points. Minimize the sum of squared vertical distances (errors) from points to the line.

Linear Algebra Connection

Normal Equations (Closed Form):

$$\mathbf{w} = (\mathbf{X}^\top \mathbf{X})^{-1} \mathbf{X}^\top \mathbf{y}$$

Geometric Interpretation: $\hat{\mathbf{y}} = \mathbf{X}(\mathbf{X}^\top \mathbf{X})^{-1} \mathbf{X}^\top \mathbf{y}$ is the projection of $\mathbf{y}$ onto column space of $\mathbf{X}$.

Gradient Descent:

$$\mathbf{w}^{(t+1)} = \mathbf{w}^{(t)} - \alpha \cdot \frac{2}{n} \mathbf{X}^\top (\mathbf{X}\mathbf{w}^{(t)} - \mathbf{y})$$

</> Step-by-Step Example

House Price Prediction:

Size (sqft)	Price ($K)
1500	300
2000	400
2500	500

Design matrix: $\mathbf{X} = \begin{bmatrix} 1 & 1500 \\ 1 & 2000 \\ 1 & 2500 \end{bmatrix}$, $\mathbf{y} = \begin{bmatrix} 300 \\ 400 \\ 500 \end{bmatrix}$

$$\mathbf{X}^\top \mathbf{X} = \begin{bmatrix} 3 & 6000 \\ 6000 & 12,500,000 \end{bmatrix}, \quad \mathbf{X}^\top \mathbf{y} = \begin{bmatrix} 1200 \\ 2,450,000 \end{bmatrix}$$

$$\mathbf{w} = (\mathbf{X}^\top \mathbf{X})^{-1} \mathbf{X}^\top \mathbf{y} = \begin{bmatrix} 0 \\ 0.2 \end{bmatrix}$$

Model: Price = 0 + 0.2 × Size ($200 per sqft)

3.2 Ridge Regression (L2 Regularization)

Intuition

Linear regression with penalty for large weights. Like keeping springs on weights to prevent them from growing too large.

Linear Algebra Connection

Tikhonov Regularization:

$$J(\mathbf{w}) = \|\mathbf{X}\mathbf{w} - \mathbf{y}\|^2 + \lambda \|\mathbf{w}\|^2$$

$$\mathbf{w}_{\text{ridge}} = (\mathbf{X}^\top \mathbf{X} + \lambda \mathbf{I})^{-1} \mathbf{X}^\top \mathbf{y}$$

SVD Interpretation: If $\mathbf{X} = \mathbf{U}\boldsymbol{\Sigma}\mathbf{V}^\top$, then:

$$\mathbf{w}_{\text{ridge}} = \mathbf{V}(\boldsymbol{\Sigma}^2 + \lambda \mathbf{I})^{-1} \boldsymbol{\Sigma} \mathbf{U}^\top \mathbf{y}$$

Shrinks small singular values more.

Step-by-Step Example

With = 1: Original: $\mathbf{X}^\top\mathbf{X} = \begin{bmatrix} 5 & 10 \\ 10 & 20 \end{bmatrix}$

Ridge: $\mathbf{X}^\top\mathbf{X} + \lambda\mathbf{I} = \begin{bmatrix} 6 & 10 \\ 10 & 21 \end{bmatrix}$

Effect on weights:

- Without ridge: $\mathbf{w} = [2.0, -1.5]$
- With ridge (=1): $\mathbf{w} = [1.8, -1.3]$ (shrunk toward zero)
- With ridge (=10): $\mathbf{w} = [0.9, -0.7]$ (more shrinkage)

All weights remain non-zero, just smaller.

3.3 Lasso Regression (L1 Regularization)

Intuition

Linear regression that can zero out unimportant weights. Like having a budget for feature importance.

Linear Algebra Connection

L1 Regularization:

$$J(\mathbf{w}) = \|\mathbf{X}\mathbf{w} - \mathbf{y}\|^2 + \lambda\|\mathbf{w}\|_1$$

Subgradient:

$$\frac{\partial |w_j|}{\partial w_j} = \begin{cases} 1 & \text{if } w_j > 0 \\ -1 & \text{if } w_j < 0 \\ [-1, 1] & \text{if } w_j = 0 \end{cases}$$

Soft Thresholding: Solution satisfies: $w_j = S_\lambda(z_j)$ where $S_\lambda(z) = \text{sign}(z)(|z| - \lambda)_+$

Step-by-Step Example

Feature Selection: Original weights: $[0.3, -0.1, 0.4, 0.05, -0.02]$
Apply Lasso (=0.1):

- $|0.3| > 0.1 \to 0.3 - 0.1 = 0.2$
- $|-0.1| = 0.1 \to -0.1 + 0.1 = 0$ (zeroed!)
- $|0.4| > 0.1 \to 0.4 - 0.1 = 0.3$
- $|0.05| < 0.1 \to 0$ (zeroed!)
- $|-0.02| < 0.1 \to 0$ (zeroed!)

Final: $[0.2, 0, 0.3, 0, 0]$ (2 features selected)

4 Dimensionality Reduction

4.1 Principal Component Analysis (PCA): Maximum Variance Directions

Intuition

Find new axes (principal components) that capture maximum variance in data. Like finding the "viewpoints" that show the most spread.

Linear Algebra Connection

Steps:

1. Center data: $\tilde{\mathbf{X}} = \mathbf{X} - \mathbf{1}\bar{\mathbf{x}}^\top$
2. Covariance: $\mathbf{C} = \frac{1}{n-1}\tilde{\mathbf{X}}^\top\tilde{\mathbf{X}}$
3. Eigen decomposition: $\mathbf{C} = \mathbf{V}\mathbf{\Lambda}\mathbf{V}^\top$
4. Project: $\mathbf{Z} = \tilde{\mathbf{X}}\mathbf{V}_k$

SVD Approach: $\mathbf{X} = \mathbf{U}\mathbf{\Sigma}\mathbf{V}^\top$, then $\mathbf{Z} = \mathbf{U}_k\mathbf{\Sigma}_k$

Step-by-Step Example

2D Data to 1D: Data: $\mathbf{X} = \begin{bmatrix} 1 & 2 \\ 2 & 3 \\ 3 & 4 \\ 4 & 5 \end{bmatrix}$

1. Center: $\tilde{\mathbf{X}} = \begin{bmatrix} -1.5 & -1.5 \\ -0.5 & -0.5 \\ 0.5 & 0.5 \\ 1.5 & 1.5 \end{bmatrix}$
2. Covariance: $\mathbf{C} = \frac{1}{3}\tilde{\mathbf{X}}^\top\tilde{\mathbf{X}} = \begin{bmatrix} 1.67 & 1.67 \\ 1.67 & 1.67 \end{bmatrix}$
3. Eigenvectors: $\mathbf{v}_1 = [0.707, 0.707]^\top$ (=3.33), $\mathbf{v}_2 = [-0.707, 0.707]^\top$ (=0)
4. Project onto PC1: $\mathbf{z} = \tilde{\mathbf{X}}\mathbf{v}_1 = [-2.12, -0.71, 0.71, 2.12]^\top$

Variance preserved: $3.33/(3.33 + 0) = 100\%$

4.2 Independent Component Analysis (ICA): Blind Source Separation

Intuition

"Cocktail party problem": Separate mixed signals into original sources. Assumes sources are statistically independent and non-Gaussian.

Linear Algebra Connection

Linear Mixing Model:

$$\mathbf{x} = \mathbf{A}\mathbf{s}$$

where $\mathbf{A}$ is mixing matrix, $\mathbf{s}$ are independent sources.
Goal: Find $\mathbf{W} \approx \mathbf{A}^{-1}$ such that $\mathbf{y} = \mathbf{W}\mathbf{x}$ are independent.
Non-Gaussianity Measures:

- Kurtosis: $\text{kurt}(y) = E[y^4] - 3(E[y^2])^2$
- Negentropy: $J(y) = H(y_{\text{gauss}}) - H(y)$

Step-by-Step Example

Separating Audio Signals: Two microphones record mixed signals:

$$x_1(t) = 0.8s_1(t) + 0.3s_2(t)$$
$$x_2(t) = 0.2s_1(t) + 0.7s_2(t)$$

Matrix form: $\mathbf{x}(t) = \begin{bmatrix} 0.8 & 0.3 \\ 0.2 & 0.7 \end{bmatrix} \mathbf{s}(t)$
ICA finds: $\mathbf{W} = \begin{bmatrix} 1.5 & -0.64 \\ -0.43 & 1.71 \end{bmatrix} \approx \mathbf{A}^{-1}$
Recovered: $\hat{\mathbf{s}}(t) = \mathbf{W}\mathbf{x}(t) \approx \mathbf{s}(t)$

5 Association Rule Mining

5.1 Apriori Algorithm: Frequent Itemset Mining

Intuition

Market basket analysis: Find items frequently bought together. Based on "if someone buys X, they likely buy Y."

Linear Algebra Connection

Binary Matrix Representation: Rows = transactions, Columns = items (1 if bought)
Metrics:

$$\text{Support}(X) = \frac{\text{count}(X)}{N}$$
$$\text{Confidence}(X \to Y) = \frac{\text{support}(X \cup Y)}{\text{support}(X)}$$
$$\text{Lift}(X \to Y) = \frac{\text{support}(X \cup Y)}{\text{support}(X)\text{support}(Y)}$$

Apriori Property: All subsets of frequent itemsets must be frequent.

</> Step-by-Step Example

Transactions
{milk, bread}
{milk}
{bread, butter}
{milk, bread, butter}

Minimum support = 50% (2 transactions)

- 1-itemsets: milk(3), bread(3), butter(2) ← all frequent
- 2-itemsets: milk+bread(2), milk+butter(1), bread+butter(2)
- Frequent: milk+bread, bread+butter

Rules:

- milk → bread: Support=2/4=0.5, Confidence=2/3=0.67
- bread → milk: Support=0.5, Confidence=2/3=0.67
- bread → butter: Support=0.5, Confidence=2/3=0.67

5.2 FP-Growth: Frequent Pattern without Candidates

Intuition

Build a compact tree structure (FP-tree) that compresses the database. Mine patterns by traversing the tree instead of scanning database repeatedly.

Linear Algebra Connection

FP-Tree Construction:

1. Scan database once, count item frequencies
2. Sort items by frequency
3. Build tree: Each node = (item, count)

Mining: For each item, construct conditional pattern base and mine recursively.
Complexity: $O(n)$ for construction vs Apriori's $O(2^d)$ candidate generation.

</> Step-by-Step Example

Same transactions as Apriori example:

1. Count: milk(3), bread(3), butter(2)
2. Order: milk, bread, butter
3. Build FP-tree:

```
Root
 |
(milk:3)
 |   \
(bread:2) (bread:1)
 |         |
(butter:1) NULL
```

4. Conditional pattern base for butter:
   - Paths: milk→bread→butter:1, bread→butter:1
   - Conditional FP-tree for butter: bread:2
   - Frequent patterns with butter: butter, bread,butter

6 Anomaly Detection

6.1 Z-score: Statistical Outlier Detection

Intuition

Flag points that are "too many standard deviations away" from the mean. Classic statistical approach.

Linear Algebra Connection

Univariate:

$$z = \frac{x - \mu}{\sigma}$$

Flag if $|z| >$ threshold (typically 2 or 3).
Multivariate (Mahalanobis Distance):

$$D_M(\mathbf{x}) = \sqrt{(\mathbf{x} - \boldsymbol{\mu})^\top \boldsymbol{\Sigma}^{-1} (\mathbf{x} - \boldsymbol{\mu})}$$

Accounts for correlations between features.

Step-by-Step Example

Server Response Times: Data: [20, 21, 22, 23, 24, 100] ms

$$\mu = \frac{20 + 21 + 22 + 23 + 24 + 100}{6} = 35$$

$$\sigma = \sqrt{\frac{(20 - 35)^2 + \cdots + (100 - 35)^2}{6}} = 32.6$$

$$z_{100} = \frac{100 - 35}{32.6} = 2.0$$

Threshold=2.5 $\rightarrow$ Not an outlier
Threshold=2.0 $\rightarrow$ Flagged as outlier
With multivariate: Consider [response$_{t}ime, CPU_{u}sage]$: $Point$: $[100, 95\%]mightbenormalifhighCPUcausesslowresponse.Mahalanobisdistanceconsidersthiscorrelation.$

6.2 Isolation Forest: Isolate Anomalies

Intuition

"Anomalies are few and different." Instead of profiling normal data, explicitly isolate anomalies through random splits. Anomalies are easier to isolate (fewer splits needed).

Linear Algebra Connection

Algorithm:

1. Build trees by randomly selecting feature and split value
2. Average path length to isolate point = anomaly score
3. Anomaly score: $s(x, n) = 2^{-\frac{E(h(x))}{c(n)}}$

where $c(n)$ is average path length of unsuccessful search in BST.
Properties:

- Sub-sampling works well (anomalies still isolated)
- Linear time complexity
- Handles high dimensions well

Step-by-Step Example

2D Data with One Anomaly: Normal: Points clustered around (0,0)
Anomaly: Point at (10,10)
Isolation process:

- Normal point: Many splits needed to isolate from cluster
- Anomaly: First split x>5 isolates it immediately!

Path lengths:

- Anomaly: 1 split (path length=1)
- Normal: 8 splits (path length=8)

Anomaly score $2^{-1/5.6} = 0.88$ (close to 1 = anomaly)
Normal score $2^{-8/5.6} = 0.37$ (close to 0 = normal)

7 Semi-Supervised Learning

7.1 Self-Training: Bootstrapping with Confidence

Intuition

Use the model's own confident predictions as additional training data. Like a student checking their own work and re-studying what they're unsure about.

Linear Algebra Connection

Pseudo-Labeling Algorithm:

1. Train on labeled data $L = \{(\mathbf{x}_i, y_i)\}$
2. Predict on unlabeled data $U = \{\mathbf{x}_j\}$
3. Select samples with prediction confidence > τ
4. Add $(\mathbf{x}_j, \hat{y}_j)$ to L
5. Repeat

Confidence Measures:

- Probability: $P(y|\mathbf{x})$
- Margin: $P(\hat{y}_1|\mathbf{x}) - P(\hat{y}_2|\mathbf{x})$
- Entropy: $-\sum P(y|\mathbf{x}) \log P(y|\mathbf{x})$

</> Step-by-Step Example

Email Classification: Initial: 100 labeled emails (50 spam, 50 not) Unlabeled: 10,000 emails
Iteration 1:

- Train logistic regression on 100 labeled
- Predict on 10,000 unlabeled
- 2,000 have spam probability > 0.9 or < 0.1
- Add these as pseudo-labeled

Iteration 2:

- Retrain on 100 + 2,000 = 2,100 emails
- Predict on remaining 8,000
- Add 1,500 more confident predictions

Continue until convergence or no more high-confidence predictions.

7.2 Co-Training: Multiple Views

Intuition

Two classifiers trained on different feature sets teach each other. Like having two experts with different specialties collaborating.

Linear Algebra Connection

Assumptions:

- Features can be split into two conditionally independent views
- Each view is sufficient for classification

Algorithm:

1. Split features: $\mathbf{x} = [\mathbf{x}^1, \mathbf{x}^2]$
2. Train classifier C_1 on $(\mathbf{x}^1, y)$, C_2 on $(\mathbf{x}^2, y)$
3. Each classifier labels unlabeled data for the other
4. Add confident predictions to each other's training set

Step-by-Step Example

Webpage Classification: Two natural views:

- View 1: Words on page (bag-of-words)
- View 2: Links pointing to page (anchor text)

Process:

1. C_1 (words classifier) labels 100 unlabeled pages
2. Adds 50 confident predictions to C_2's training data
3. C_2 (links classifier) labels 100 unlabeled pages
4. Adds 60 confident predictions to C_1's training data
5. Both classifiers improve using each other's expertise

8 Reinforcement Learning

8.1 Q-Learning: Model-Free Value Learning

Intuition

Learn a "cheat sheet" (Q-table) that tells you how good each action is in each state. Update estimates based on actual rewards received.

Linear Algebra Connection

Bellman Equation:

$$Q(s,a) \leftarrow Q(s,a) + \alpha[r + \gamma \max_{a'} Q(s',a') - Q(s,a)]$$

Matrix Form: Q-table is $|S| \times |A|$ matrix updated via:

$$\mathbf{Q} \leftarrow \mathbf{Q} + \alpha(\mathbf{r} + \gamma \max(\mathbf{Q}') - \mathbf{Q})$$

Parameters:

- α: Learning rate (0.1 typical)
- γ: Discount factor (0.9 typical)
- ϵ: Exploration rate (decays over time)

</> Step-by-Step Example

Grid World (3×3): States: 9 positions, Actions: up, down, left, right Goal: Reach center (reward=+10), Walls give -1
Initial Q-table (all zeros):

State	Up	Down	Left	Right
(0,0)	0	0	0	0
(0,1)	0	0	0	0
...	...	...	...	...

Episode: Start at (0,0), choose right (exploration)

- $s = (0, 0)$, $a =$ right, $r = -1$ (hit wall), $s' = (0, 0)$
- Update: $Q((0, 0), \text{right}) = 0 + 0.1[-1 + 0.90 - 0] = -0.1$

Next: Choose down from (0,0)

- $s = (0, 0)$, $a =$ down, $r = 0$, $s' = (1, 0)$
- $Q((0, 0), \text{down}) = 0 + 0.1[0 + 0.90 - 0] = 0$

8.2 Policy Gradient Methods: Direct Policy Optimization

Intuition

Directly adjust the policy parameters to increase expected reward. Like tuning a radio dial to get clearer signal.

Linear Algebra Connection

REINFORCE Algorithm:

$$\nabla J(\theta) = E[\sum_{t=0}^{T} \nabla_\theta \log \pi_\theta(a_t|s_t) G_t]$$

$$\theta \leftarrow \theta + \alpha G_t \nabla_\theta \log \pi_\theta(a_t|s_t)$$

Advantage Actor-Critic:

$$\theta \leftarrow \theta + \alpha A(s_t, a_t) \nabla_\theta \log \pi_\theta(a_t|s_t)$$

where $A(s, a) = Q(s, a) - V(s)$ is advantage function.

</> Step-by-Step Example

CartPole Balancing: State: [cart position, cart velocity, pole angle, pole velocity] Action: push left (0) or right (1)
Policy network: $\pi_\theta(a|s) = \sigma(\theta^\top \phi(s))$

- Start with random θ
- Generate trajectory: $s_0, a_0, r_0, s_1, a_1, r_1, \ldots$
- Compute returns: $G_t = \sum_{k=t}^{T} \gamma^{k-t} r_k$
- Update: $\theta \leftarrow \theta + \alpha \sum G_t \nabla \log \pi(a_t|s_t)$

Example step:

$$\begin{aligned} \pi(\text{right}|s) &= 0.7, \text{ chose right} \\ \log \pi &= \log(0.7) = -0.357 \\ G_t &= 10.0 \\ \nabla_\theta \log \pi &= \phi(s) \quad \text{(for logistic policy)} \\ \theta &\leftarrow \theta + 0.01 \times 10.0 \times \phi(s) \end{aligned}$$

Appendix: Linear Algebra Quick Reference

Algorithm Summary

- **Dot Product:** $\mathbf{a} \cdot \mathbf{b} = \sum a_i b_i$
- **Matrix Mult:** $(\mathbf{AB})_{ij} = \sum_k A_{ik} B_{kj}$
- **Transpose:** $(\mathbf{A}^\top)_{ij} = A_{ji}$
- **Inverse:** $\mathbf{A}^{-1}\mathbf{A} = \mathbf{I}$
- **Determinant:** $\det(\mathbf{A})$, volume scaling
- **Eigenvalues:** $\mathbf{Av} = \lambda \mathbf{v}$
- **SVD:** $\mathbf{A} = \mathbf{U\Sigma V}^\top$
- **Norms:**
  - L1: $\|\mathbf{x}\|_1 = \sum |x_i|$
  - L2: $\|\mathbf{x}\|_2 = \sqrt{\sum x_i^2}$
  - L: $\|\mathbf{x}\|_\infty = \max |x_i|$
- **Gradient:** $\nabla f = [\frac{\partial f}{\partial x_1}, \ldots]$
- **Hessian:** $\mathbf{H}_{ij} = \frac{\partial^2 f}{\partial x_i \partial x_j}$

Common ML Operations as Linear Algebra

ML Operation	Linear Algebra Form
Forward Propagation	$\mathbf{Z}^{(l)} = \mathbf{A}^{(l-1)}\mathbf{W}^{(l)} + \mathbf{b}^{(l)}$
Backpropagation	$\delta^{(l)} = (\delta^{(l+1)}\mathbf{W}^{(l+1)\top}) \odot \sigma'(Z^{(l)})$
PCA	$\mathbf{Z} = \tilde{\mathbf{X}}\mathbf{V}_k$ where $\mathbf{C} = \mathbf{V\Lambda V}^\top$
K-means	Assign: $c_i = \arg\min_j \|\mathbf{x}_i - \boldsymbol{\mu}_j\|^2$
SVM Dual	$\max_\alpha \sum \alpha_i - \frac{1}{2} \sum\sum \alpha_i \alpha_j y_i y_j \mathbf{x}_i \cdot \mathbf{x}_j$

Conclusion

Intuition

Every machine learning algorithm is a specific combination of linear algebra operations designed to extract patterns from data. Understanding these mathematical foundations allows you to choose the right tool for each problem and implement custom solutions when needed.

Remember:

1. **Data as Vectors** - Each sample is a point in high-D space
2. **Learning as Optimization** - Find parameters minimizing loss
3. **Prediction as Transformation** - Apply learned function to new data
4. **Evaluation as Distance** - Compare predictions to truth
